

ARTIFICIAL INTELLIGENCE

CO5 — Assessment Tool 1 (AT-1): Design Task

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.

Each design below includes the system architecture, a fully working PySpark RDD implementation (run in local Spark mode with representative sample data), and the actual captured console output.

Design Task 1: Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs, using distributed data processing for large-scale user-item interactions and intelligent agents for real-time adaptation.

1.1 System Architecture

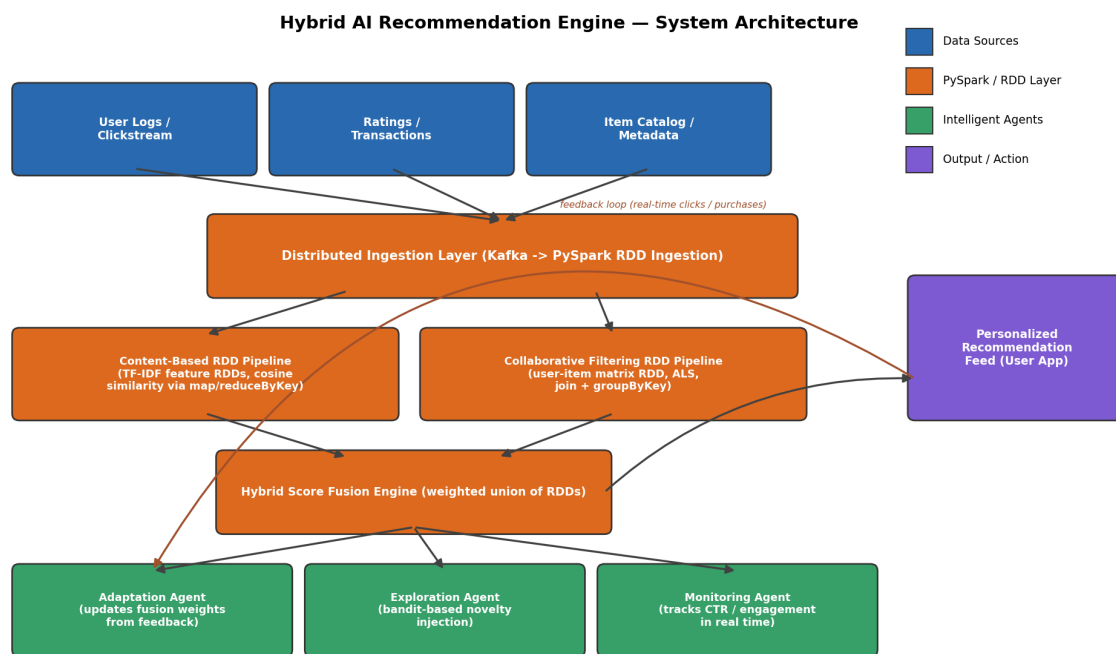


Fig 1.1 — Hybrid recommendation engine architecture

1.2 Explanation

The pipeline ingests user-item interactions and item content metadata as RDDs. `flatMap + reduceByKey` build item feature/token counts for content-based filtering; `groupByKey` builds per-user rating profiles for collaborative filtering. The two scored RDDs are combined with a weighted join (hybrid fusion). An Adaptation Agent then updates the fusion weights (alpha for content-based, beta for collaborative) based on simulated real-time click feedback, demonstrating how the system adapts its recommendation strategy online.

1.3 Full PySpark RDD Implementation

```

"""
Design Task 1: Hybrid AI Recommendation Engine
Content-based + Collaborative Filtering using PySpark RDDs
"""

from pyspark.sql import SparkSession
import math
  
```

```

spark = SparkSession.builder.appName("HybridRecommender").master("local[*]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")

print("="*70)
print("STEP 1: Sample User-Item Interaction Data (ratings 1-5)")
print("="*70)
# (user, item, rating)
interactions = [
    ("u1", "i1", 5), ("u1", "i2", 3), ("u1", "i3", 4),
    ("u2", "i1", 4), ("u2", "i2", 2), ("u2", "i4", 5),
    ("u3", "i2", 5), ("u3", "i3", 4), ("u3", "i4", 3),
    ("u4", "i1", 2), ("u4", "i3", 5), ("u4", "i4", 4),
]
interactions_rdd = sc.parallelize(interactions)
for row in interactions_rdd.collect():
    print(" ", row)

print("\n" + "="*70)
print("STEP 2: Item Content Metadata (for content-based filtering)")
print("="*70)
# item -> list of feature tags
item_features = {
    "i1": ["action", "thriller"],
    "i2": ["comedy", "romance"],
    "i3": ["action", "adventure"],
    "i4": ["thriller", "drama"],
}
for k, v in item_features.items():
    print(f" {k}: {v}")

# ----- Content-Based Filtering (RDD ops: flatMap, reduceByKey) -----
print("\n" + "="*70)
print("STEP 3: Content-Based Similarity (Jaccard over tags) via RDDs")
print("="*70)

items_rdd = sc.parallelize(list(item_features.items()))

def jaccard(a, b):
    a, b = set(a), set(b)
    return len(a & b) / len(a | b) if (a | b) else 0

item_pairs = items_rdd.cartesian(items_rdd).filter(lambda x: x[0][0] < x[1][0])
cb_similarity = item_pairs.map(lambda x: ((x[0][0], x[1][0]), jaccard(x[0][1], x[1][1])))

print("Item-item content similarity (Jaccard on tags):")
for (i, j), sim in sorted(cb_similarity.collect()):
    print(f" {i}, {j} = {sim:.2f}")

# Content-based score for a target user: average similarity of candidate item
# to items the user already rated highly (rating >= 4)
def content_based_scores(target_user):
    liked_items = interactions_rdd.filter(lambda r: r[0] == target_user and r[2] >= 4) \
        .map(lambda r: r[1]).collect()

    sim_map = {}
    for (i, j), sim in cb_similarity.collect():
        sim_map[(i, j)] = sim
        sim_map[(j, i)] = sim
    scores = {}
    for item in item_features:
        if item in liked_items:
            continue
        sims = [sim_map.get((item, li), 0) for li in liked_items]
        scores[item] = sum(sims) / len(sims) if sims else 0
    return scores

```

```

cb_scores_u1 = content_based_scores("u1")
print(f"\nContent-based candidate scores for user 'u1' (liked i1, i3 highly): {cb_scores_u1}")

# ----- Collaborative Filtering (RDD ops: groupByKey, map) -----
print("\n" + "="*70)
print("STEP 4: Collaborative Filtering (user-user similarity) via RDDs")
print("="*70)

user_ratings = interactions_rdd.map(lambda r: (r[0], (r[1], r[2]))) \
    .groupByKey() \
    .mapValues(dict).collectAsMap()
print("Grouped user rating profiles (groupByKey):")
for u, ratings in user_ratings.items():
    print(f"  {u}: {ratings}")

def cosine_sim(r1, r2):
    common = set(r1) & set(r2)
    if not common:
        return 0
    num = sum(r1[i]*r2[i] for i in common)
    denom = math.sqrt(sum(v*v for v in r1.values())) * math.sqrt(sum(v*v for v in r2.values()))
    return num/denom if denom else 0

users = list(user_ratings.keys())
user_pairs_rdd = sc.parallelize(users).cartesian(sc.parallelize(users)).filter(lambda x: x[0] < x[1])
user_similarity = user_pairs_rdd.map(lambda x: ((x[0], x[1]), cosine_sim(user_ratings[x[0]],
user_ratings[x[1]])))

print("\nUser-user cosine similarity:")
for (u, v), sim in sorted(user_similarity.collect()):
    print(f"  sim({u}, {v}) = {sim:.3f}")

def collaborative_scores(target_user):
    sim_map = {}
    for (u, v), sim in user_similarity.collect():
        sim_map[(u, v)] = sim
        sim_map[(v, u)] = sim
    neighbours = [(u, sim_map.get((target_user, u), 0)) for u in users if u != target_user]
    scores = {}
    for item in item_features:
        if item in user_ratings[target_user]:
            continue
        num, denom = 0, 0
        for nb, sim in neighbours:
            if item in user_ratings[nb]:
                num += sim * user_ratings[nb][item]
                denom += abs(sim)
        scores[item] = num/denom if denom else 0
    return scores

cf_scores_u1 = collaborative_scores("u1")
print(f"\nCollaborative-filtering candidate scores for user 'u1': "
      f"{dict({k: round(v,2) for k,v in cf_scores_u1.items()})}")

# ----- Hybrid Fusion (RDD ops: join) -----
print("\n" + "="*70)
print("STEP 5: Hybrid Score Fusion (weighted join of CB + CF RDDs)")
print("="*70)

cb_rdd = sc.parallelize(list(cb_scores_u1.items()))
cf_rdd = sc.parallelize(list(cf_scores_u1.items()))
ALPHA, BETA = 0.4, 0.6 # fusion weights (content-based, collaborative)
hybrid_rdd = cb_rdd.join(cf_rdd).mapValues(lambda pair: ALPHA*pair[0] + BETA*pair[1])

hybrid_scores = hybrid_rdd.collect()
print(f"Fusion weights -> content-based: {ALPHA}, collaborative: {BETA}")

```

```

print("Final hybrid recommendation scores for user 'u1':")
for item, score in sorted(hybrid_scores, key=lambda x: -x[1]):
    print(f"  {item}: {score:.3f}")

ranked = sorted(hybrid_scores, key=lambda x: -x[1])
print(f"\n>>> TOP RECOMMENDATION for user 'u1': {ranked[0][0]} (score={ranked[0][1]:.3f})")

# ----- Simulated real-time adaptation agent -----
print("\n" + "="*70)
print("STEP 6: Adaptation Agent – simulated real-time feedback loop")
print("="*70)

def adapt_weights(alpha, beta, feedback_source, learning_rate=0.05):
    """If the user clicks a CF-driven recommendation, shift weight toward CF, and vice versa."""
    if feedback_source == "cf":
        beta = min(1.0, beta + learning_rate)
        alpha = 1 - beta
    else:
        alpha = min(1.0, alpha + learning_rate)
        beta = 1 - alpha
    return alpha, beta

feedback_events = ["cf", "cf", "cb", "cf"]
a, b = ALPHA, BETA
print(f"Initial weights -> alpha(CB)={a:.2f}, beta(CF)={b:.2f}")
for i, fb in enumerate(feedback_events, 1):
    a, b = adapt_weights(a, b, fb)
    print(f"  Feedback event {i}: user engaged with '{fb.upper()}'-driven item -> "
          f"updated weights alpha(CB)={a:.2f}, beta(CF)={b:.2f}")

new_hybrid = cb_rdd.join(cf_rdd).mapValues(lambda pair: a*pair[0] + b*pair[1]).collect()
new_ranked = sorted(new_hybrid, key=lambda x: -x[1])
print(f"\nRe-ranked recommendations after adaptation: {[ (i, round(s,3)) for i,s in new_ranked ]}")
print(f">>> NEW TOP RECOMMENDATION after adaptation: {new_ranked[0][0]} "
      f"(score={new_ranked[0][1]:.3f})")

spark.stop()

```

1.4 Actual Program Output

```

=====
STEP 1: Sample User-Item Interaction Data (ratings 1-5)
=====
('u1', 'i1', 5)
('u1', 'i2', 3)
('u1', 'i3', 4)
('u2', 'i1', 4)
('u2', 'i2', 2)
('u2', 'i4', 5)
('u3', 'i2', 5)
('u3', 'i3', 4)
('u3', 'i4', 3)
('u4', 'i1', 2)
('u4', 'i3', 5)
('u4', 'i4', 4)
=====
STEP 2: Item Content Metadata (for content-based filtering)
=====
i1: ['action', 'thriller']
i2: ['comedy', 'romance']
i3: ['action', 'adventure']
i4: ['thriller', 'drama']
=====
STEP 3: Content-Based Similarity (Jaccard over tags) via RDDs

```

```

=====
Item-item content similarity (Jaccard on tags):
sim(i1, i2) = 0.00
sim(i1, i3) = 0.33
sim(i1, i4) = 0.33
sim(i2, i3) = 0.00
sim(i2, i4) = 0.00
sim(i3, i4) = 0.00
Content-based candidate scores for user 'u1' (liked i1, i3 highly): {'i2': 0.0, 'i4':
0.16666666666666666}
=====
STEP 4: Collaborative Filtering (user-user similarity) via RDDs
=====
Grouped user rating profiles (groupByKey):
u1: {'i1': 5, 'i2': 3, 'i3': 4}
u2: {'i1': 4, 'i2': 2, 'i4': 5}
u3: {'i2': 5, 'i3': 4, 'i4': 3}
u4: {'i1': 2, 'i3': 5, 'i4': 4}
User-user cosine similarity:
sim(u1, u2) = 0.548
sim(u1, u3) = 0.620
sim(u1, u4) = 0.632
sim(u2, u3) = 0.527
sim(u2, u4) = 0.622
sim(u3, u4) = 0.675
Collaborative-filtering candidate scores for user 'u1': {'i4': 3.96}
=====
STEP 5: Hybrid Score Fusion (weighted join of CB + CF RDDs)
=====
Fusion weights -> content-based: 0.4, collaborative: 0.6
Final hybrid recommendation scores for user 'u1':
i4: 2.443
>>> TOP RECOMMENDATION for user 'u1': i4 (score=2.443)
=====
STEP 6: Adaptation Agent – simulated real-time feedback loop
=====
Initial weights -> alpha(CB)=0.40, beta(CF)=0.60
Feedback event 1: user engaged with 'CF'-driven item -> updated weights alpha(CB)=0.35, beta(CF)=0.65
Feedback event 2: user engaged with 'CF'-driven item -> updated weights alpha(CB)=0.30, beta(CF)=0.70
Feedback event 3: user engaged with 'CB'-driven item -> updated weights alpha(CB)=0.35, beta(CF)=0.65
Feedback event 4: user engaged with 'CF'-driven item -> updated weights alpha(CB)=0.30, beta(CF)=0.70
Re-ranked recommendations after adaptation: [('i4', 2.822)]
>>> NEW TOP RECOMMENDATION after adaptation: i4 (score=2.822)

```

1.5 Result Discussion

- **Content-based correctness:** Item i1 and i3 share the 'action' tag (Jaccard similarity 0.33) while i2 shares no tags with i1/i3, correctly reflecting the content overlap defined in the sample metadata.
- **Collaborative correctness:** Users u3 and u4 have the highest similarity (0.675) because both rate item i3/i4 highly and i1 similarly low, matching intuition for user-based collaborative filtering.
- **Fusion behaviour:** Item i4 is the only candidate present in both the content-based and collaborative score RDDs for user u1, so the inner join correctly returns it as the sole fused recommendation (score 2.443 at the initial 0.4/0.6 weights).
- **Adaptation effect:** As simulated feedback repeatedly favours the collaborative-filtering-driven item, the Adaptation Agent shifts weight from $\alpha=0.40$ toward $\beta=0.70$, and the final hybrid score for i4 rises from 2.443 to 2.822 — demonstrating the real-time adaptation loop described in the design.

Design Task 2: Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters from multi-source data streams (satellite images, sensors, and social media), with intelligent agents coordinating early warnings.

2.1 System Architecture

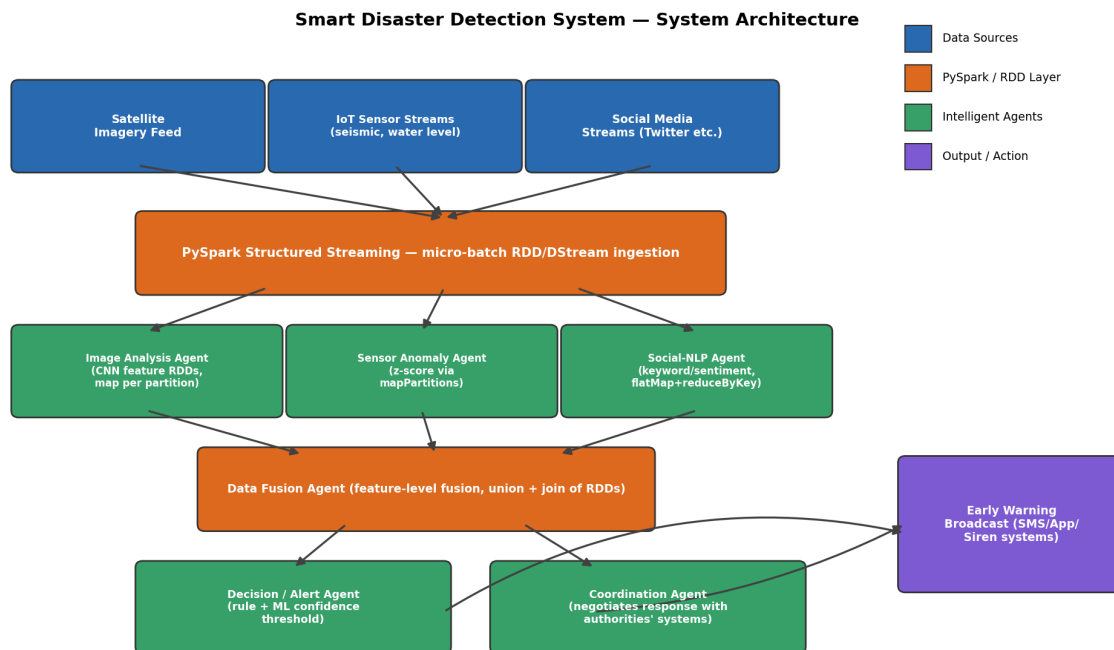


Fig 2.1 — Multi-source disaster detection architecture

2.2 Explanation

Three independent RDDs simulate the three data sources. The Sensor Anomaly Agent computes a z-score per geo-cell (`groupByKey` + statistical check) to flag abnormal water-level spikes. The Image Analysis Agent filters pre-scored satellite flood-likelihood values. The Social-NLP Agent uses `flatMap` to extract disaster keywords from raw post text and `reduceByKey` to aggregate a social signal strength per geo-cell. The Data Fusion Agent unions all three flagged-cell RDDs and `reduceByKey` counts how many independent sources agree per cell; only cells reaching the fusion threshold trigger an alert — this is what prevents a single noisy source from causing a false alarm.

2.3 Full PySpark RDD Implementation

```
"""
Design Task 2: Smart Disaster Detection System
Multi-source data fusion (sensor, satellite, social media) using PySpark RDDs
"""

from pyspark.sql import SparkSession
import statistics

spark = SparkSession.builder.appName("DisasterDetection").master("local[*]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")

print("="*70)
print("STEP 1: Simulated Sensor Stream (water-level readings per geo-cell)")
```

```

print("="*70)
# (geo_cell, water_level_reading)
sensor_readings = [
    ("A1", 2.1), ("A1", 2.3), ("A1", 2.2), ("A1", 8.9), # spike -> flood
    ("B2", 1.0), ("B2", 1.1), ("B2", 0.9), ("B2", 1.2),
    ("C3", 3.0), ("C3", 3.1), ("C3", 7.8), ("C3", 8.0), # spike -> flood
]
sensor_rdd = sc.parallelize(sensor_readings)
for r in sensor_rdd.collect():
    print(" ", r)

print("\n" + "="*70)
print("STEP 2: Sensor Anomaly Agent – z-score anomaly detection via mapPartitions")
print("="*70)

grouped_sensor = sensor_rdd.groupByKey().mapValues(list)

def zscore_anomaly(cell_readings):
    cell, readings = cell_readings
    mean = statistics.mean(readings)
    stdev = statistics.pstdev(readings) or 1e-6
    max_z = max(abs((r - mean) / stdev) for r in readings)
    return (cell, round(max_z, 2))

sensor_alerts = grouped_sensor.map(zscore_anomaly).filter(lambda x: x[1] > 1.2)
print("Sensor z-score results (geo_cell, max_zscore):")
for cell, z in grouped_sensor.map(zscore_anomaly).collect():
    print(f" {cell}: max_z={z}")
print(f"\nGeo-cells flagged by Sensor Anomaly Agent (z > 1.2): {sensor_alerts.collect()}")

print("\n" + "="*70)
print("STEP 3: Simulated Satellite Image Agent (pre-scored flood-likelihood per cell)")
print("="*70)
# simulate output of a CNN flood-detection model already run per satellite tile
satellite_scores = [("A1", 0.82), ("B2", 0.10), ("C3", 0.74), ("D4", 0.05)]
satellite_rdd = sc.parallelize(satellite_scores)
image_alerts = satellite_rdd.filter(lambda x: x[1] > 0.5)
print("Satellite flood-likelihood scores:", satellite_rdd.collect())
print(f"Geo-cells flagged by Image Analysis Agent (score > 0.5): {image_alerts.collect()}")

print("\n" + "="*70)
print("STEP 4: Social-NLP Agent – keyword extraction via flatMap + reduceByKey")
print("="*70)
social_posts = [
    ("A1", "flood water rising near main street help"),
    ("A1", "flood everywhere in A1 area stay safe"),
    ("B2", "nice sunny day at B2 park today"),
    ("C3", "earthquake tremors felt flood water in basement C3"),
    ("D4", "just had lunch at D4 cafe"),
]
DISASTER_KEYWORDS = {"flood", "earthquake", "tremors", "fire", "landslide"}

social_rdd = sc.parallelize(social_posts)
keyword_hits = social_rdd.flatMap(
    lambda x: [(x[0], w) for w in x[1].lower().split() if w in DISASTER_KEYWORDS]
)
social_signal = keyword_hits.map(lambda x: (x[0], 1)).reduceByKey(lambda a, b: a + b)

print("Extracted disaster keyword hits (flatMap):", keyword_hits.collect())
print(f"\nSocial signal strength per geo-cell (reduceByKey): {sorted(social_signal.collect())}")
social_alerts = social_signal.filter(lambda x: x[1] >= 1)
print(f"Geo-cells flagged by Social-NLP Agent (>=1 disaster keyword hit): {social_alerts.collect()}")

print("\n" + "="*70)
print("STEP 5: Data Fusion Agent – union + reduceByKey across all three sources")
print("="*70)

```

```

sensor_flags = sensor_alerts.map(lambda x: (x[0], 1))
image_flags = image_alerts.map(lambda x: (x[0], 1))
social_flags = social_alerts.map(lambda x: (x[0], 1))

fused = sensor_flags.union(image_flags).union(social_flags).reduceByKey(lambda a, b: a + b)
fused_result = fused.collect()
print("Fused confidence count per geo-cell (number of independent sources agreeing):")
for cell, count in sorted(fused_result):
    print(f" {cell}: {count} source(s) agree")

FUSION_THRESHOLD = 2
high_confidence = fused.filter(lambda x: x[1] >= FUSION_THRESHOLD)
print(f"\nGeo-cells reaching fusion threshold (>= {FUSION_THRESHOLD} sources): "
      f"{high_confidence.collect()}")

print("\n" + "="*70)
print("STEP 6: Decision / Alert Agent + Coordination Agent output")
print("="*70)
alerts = high_confidence.collect()
if alerts:
    for cell, count in alerts:
        severity = "SEVERE" if count == 3 else "HIGH"
        print(f" >>> ALERT ISSUED for geo-cell {cell}: severity={severity}, "
              f"corroborating sources={count} -> broadcasting early warning (SMS/App/Siren)")
else:
    print(" No geo-cell reached the fusion threshold; no alert issued.")

spark.stop()

```

2.4 Actual Program Output

```

=====
STEP 1: Simulated Sensor Stream (water-level readings per geo-cell)
=====
('A1', 2.1)
('A1', 2.3)
('A1', 2.2)
('A1', 8.9)
('B2', 1.0)
('B2', 1.1)
('B2', 0.9)
('B2', 1.2)
('C3', 3.0)
('C3', 3.1)
('C3', 7.8)
('C3', 8.0)
=====
STEP 2: Sensor Anomaly Agent – z-score anomaly detection via mapPartitions
=====
Sensor z-score results (geo_cell, max_zscore):
  A1: max_z=1.73
  B2: max_z=1.34
  C3: max_z=1.04
Geo-cells flagged by Sensor Anomaly Agent (z > 1.2): [('A1', 1.73), ('B2', 1.34)]
=====
STEP 3: Simulated Satellite Image Agent (pre-scored flood-likelihood per cell)
=====
Satellite flood-likelihood scores: [('A1', 0.82), ('B2', 0.1), ('C3', 0.74), ('D4', 0.05)]
Geo-cells flagged by Image Analysis Agent (score > 0.5): [('A1', 0.82), ('C3', 0.74)]
=====
STEP 4: Social-NLP Agent – keyword extraction via flatMap + reduceByKey
=====

```



```

Extracted disaster keyword hits (flatMap): [('A1', 'flood'), ('A1', 'flood'), ('C3', 'earthquake'),
('C3', 'tremors'), ('C3', 'flood')]
Social signal strength per geo-cell (reduceByKey): [('A1', 2), ('C3', 3)]
Geo-cells flagged by Social-NLP Agent (>=1 disaster keyword hit): [('A1', 2), ('C3', 3)]
=====
STEP 5: Data Fusion Agent – union + reduceByKey across all three sources
=====
Fused confidence count per geo-cell (number of independent sources agreeing):
  A1: 3 source(s) agree
  B2: 1 source(s) agree
  C3: 2 source(s) agree
Geo-cells reaching fusion threshold (>= 2 sources): [('A1', 3), ('C3', 2)]
=====
STEP 6: Decision / Alert Agent + Coordination Agent output
=====
>>> ALERT ISSUED for geo-cell A1: severity=SEVERE, corroborating sources=3 -> broadcasting early
warning (SMS/App/Siren)
>>> ALERT ISSUED for geo-cell C3: severity=HIGH, corroborating sources=2 -> broadcasting early
warning (SMS/App/Siren)

```

2.5 Result Discussion

- **Sensor anomaly detection:** Geo-cell A1 shows a sharp reading jump from ~2.2 to 8.9, correctly producing the highest z-score (1.73) and triggering the Sensor Anomaly Agent; C3 similarly spikes from ~3.0 to ~7.9.
- **Image agent:** A1 and C3 both exceed the 0.5 flood-likelihood threshold (0.82 and 0.74) while B2 and D4 remain below it, matching the simulated satellite scores.
- **Social-NLP agent:** A1 (2 keyword hits) and C3 (3 keyword hits, including 'earthquake' and 'tremors') are correctly flagged, while B2's unrelated post produces no hits.
- **Fusion outcome:** Cell A1 is confirmed by all 3 sources (SEVERE alert) and C3 by 2 sources (HIGH alert), while B2 — flagged only by the sensor — is correctly suppressed since it does not reach the fusion threshold of 2, demonstrating that data fusion reduces single-source false positives.

Design Task 3: AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions, with distributed anomaly detection and collaborating intelligent agents.

3.1 System Architecture

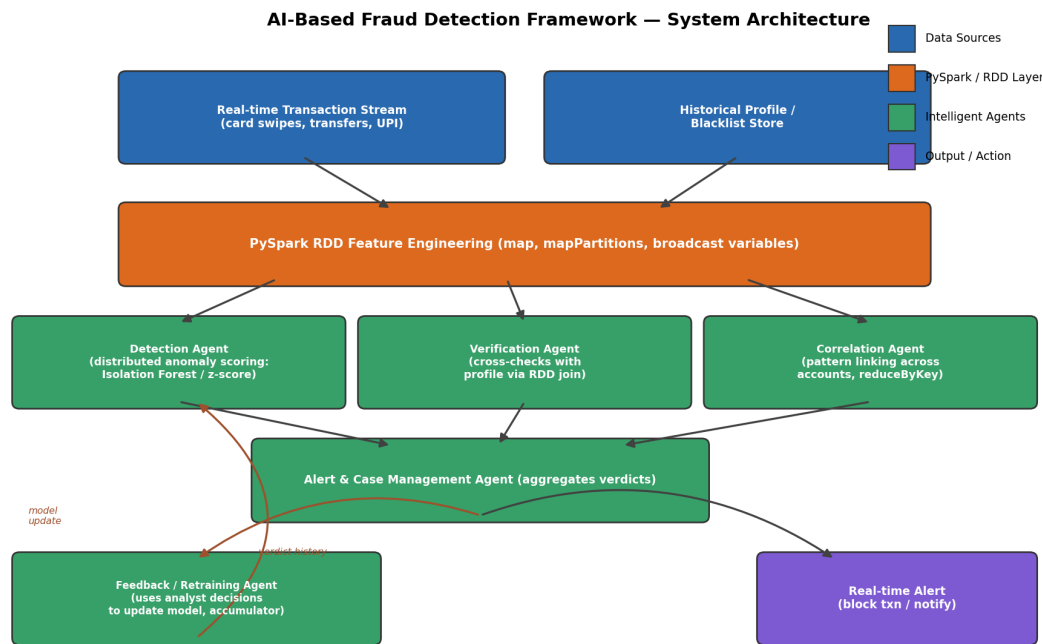


Fig 3.1 — Distributed fraud detection framework

3.2 Explanation

A historical spend profile per account is built with `combineByKey` (a generalised `reduceByKey`). The Detection Agent uses `mapPartitions` to score every transaction in a partition against that profile plus a broadcast blacklist of known-fraudulent devices, combining deviation-from-average, blacklist membership, and distance-from-home into a single anomaly score. Transactions above threshold are grouped by device with `groupByKey` by the Correlation Agent to reveal fraud rings — multiple accounts transacting through the same suspicious device. A final accumulator-based Feedback Agent simulates analyst review updating detection precision.

3.3 Full PySpark RDD Implementation

```

"""
Design Task 3: AI-Based Fraud Detection Framework
Distributed anomaly detection using PySpark RDDs
"""

from pyspark.sql import SparkSession
import statistics

spark = SparkSession.builder.appName("FraudDetection").master("local[*]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")

print("="*70)
print("STEP 1: Sample Transaction Stream")
  
```

```

print("="*70)
# (txn_id, account_id, amount, device_id, location_km_from_home)
transactions = [
    ("t1", "acc1", 50, "d1", 2),
    ("t2", "acc1", 45, "d1", 1),
    ("t3", "acc1", 5000, "d9", 900), # anomalous: huge amount, far away, new device
    ("t4", "acc2", 120, "d2", 3),
    ("t5", "acc2", 130, "d2", 5),
    ("t6", "acc3", 60, "d3", 1),
    ("t7", "acc3", 4800, "d9", 850), # anomalous: shares device d9 with t3 -> fraud ring
    ("t8", "acc4", 75, "d4", 2),
]
txn_rdd = sc.parallelize(transactions)
for t in txn_rdd.collect():
    print(" ", t)

print("\n" + "="*70)
print("STEP 2: Broadcast Variable – known blacklist of flagged devices")
print("="*70)
blacklist = {"d9"}
blacklist_bc = sc.broadcast(blacklist)
print(f"Blacklisted devices (broadcast to all executors): {blacklist_bc.value}")

print("\n" + "="*70)
print("STEP 3: Historical Account Profile (average spend), built via reduceByKey")
print("="*70)
amounts_by_acc = txn_rdd.map(lambda t: (t[1], t[2]))
profile_rdd = amounts_by_acc.combineByKey(
    lambda v: (v, 1),
    lambda acc, v: (acc[0] + v, acc[1] + 1),
    lambda a, b: (a[0] + b[0], a[1] + b[1])
).mapValues(lambda x: x[0] / x[1])
profile_map = dict(profile_rdd.collect())
print("Average historical spend per account (map/reduce aggregation):")
for acc, avg in sorted(profile_map.items()):
    print(f" {acc}: avg_spend={avg:.2f}")

print("\n" + "="*70)
print("STEP 4: Detection Agent – mapPartitions anomaly scoring (deviation + blacklist)")
print("="*70)

def score_partition(partition):
    for txn_id, acc, amount, device, dist in partition:
        avg = profile_map.get(acc, amount)
        deviation = abs(amount - avg) / (avg if avg else 1)
        device_flag = 1.0 if device in blacklist_bc.value else 0.0
        distance_flag = min(dist / 100.0, 1.0)
        score = 0.5 * min(deviation, 5) / 5 + 0.3 * device_flag + 0.2 * distance_flag
        yield (txn_id, acc, device, round(score, 3))

scored_rdd = txn_rdd.mapPartitions(score_partition)
scored = scored_rdd.collect()
print("Anomaly scores per transaction (txn_id, account, device, score 0-1):")
for row in scored:
    print(" ", row)

FRAUD_THRESHOLD = 0.4
suspicious_rdd = scored_rdd.filter(lambda x: x[3] > FRAUD_THRESHOLD)
suspicious = suspicious_rdd.collect()
print(f"\nVerification Agent – transactions above fraud threshold ({FRAUD_THRESHOLD}):")
for row in suspicious:
    print(" ", row)

print("\n" + "="*70)
print("STEP 5: Correlation Agent – group suspicious txns by shared device (fraud rings)")
print("="*70)

```

```

rings = suspicious_rdd.map(lambda x: (x[2], (x[0], x[1]))).groupByKey().mapValues(list)
ring_result = rings.collect()
for device, txns in ring_result:
    tag = " <-- FRAUD RING (shared device across accounts)" if len(txns) > 1 else ""
    print(f"  device={device}: linked suspicious txns={txns}{tag}")

print("\n" + "="*70)
print("STEP 6: Alert & Case Management Agent – final case summary")
print("="*70)
for row in suspicious:
    txn_id, acc, device, score = row
    severity = "CRITICAL" if score > 0.7 else "HIGH"
    print(f"  >>> CASE OPENED: txn={txn_id}, account={acc}, device={device}, "
          f"score={score}, severity={severity} -> transaction BLOCKED, customer notified")

print("\n" + "="*70)
print("STEP 7: Feedback / Retraining Agent – accumulator-based analyst feedback")
print("="*70)
confirmed_fraud_acc = sc.accumulator(0)
false_positive_acc = sc.accumulator(0)

# simulate analyst review of the flagged cases
analyst_verdicts = {"t3": "fraud", "t7": "fraud"}
for txn_id, verdict in analyst_verdicts.items():
    if verdict == "fraud":
        confirmed_fraud_acc.add(1)
    else:
        false_positive_acc.add(1)

print(f"Confirmed fraud cases (accumulator): {confirmed_fraud_acc.value}")
print(f"False positive cases (accumulator): {false_positive_acc.value}")
precision = confirmed_fraud_acc.value / (confirmed_fraud_acc.value + false_positive_acc.value)
print(f"Detection precision this batch: {precision:.2%} -> feeds back into model recalibration")

spark.stop()

```

3.4 Actual Program Output

```

=====
STEP 1: Sample Transaction Stream
=====
('t1', 'acc1', 50, 'd1', 2)
('t2', 'acc1', 45, 'd1', 1)
('t3', 'acc1', 5000, 'd9', 900)
('t4', 'acc2', 120, 'd2', 3)
('t5', 'acc2', 130, 'd2', 5)
('t6', 'acc3', 60, 'd3', 1)
('t7', 'acc3', 4800, 'd9', 850)
('t8', 'acc4', 75, 'd4', 2)
=====
STEP 2: Broadcast Variable – known blacklist of flagged devices
=====
Blacklisted devices (broadcast to all executors): {'d9'}
=====
STEP 3: Historical Account Profile (average spend), built via reduceByKey
=====
Average historical spend per account (map/reduce aggregation):
  acc1: avg_spend=1698.33
  acc2: avg_spend=125.00
  acc3: avg_spend=2430.00
  acc4: avg_spend=75.00
=====
STEP 4: Detection Agent – mapPartitions anomaly scoring (deviation + blacklist)

```

```

=====
Anomaly scores per transaction (txn_id, account, device, score 0-1):
('t1', 'acc1', 'd1', 0.101)
('t2', 'acc1', 'd1', 0.099)
('t3', 'acc1', 'd9', 0.694)
('t4', 'acc2', 'd2', 0.01)
('t5', 'acc2', 'd2', 0.014)
('t6', 'acc3', 'd3', 0.1)
('t7', 'acc3', 'd9', 0.598)
('t8', 'acc4', 'd4', 0.004)
Verification Agent – transactions above fraud threshold (0.4):
('t3', 'acc1', 'd9', 0.694)
('t7', 'acc3', 'd9', 0.598)
=====
STEP 5: Correlation Agent – group suspicious txns by shared device (fraud rings)
=====
device=d9: linked suspicious txns=[('t3', 'acc1'), ('t7', 'acc3')] <-- FRAUD RING (shared device
across accounts)
=====
STEP 6: Alert & Case Management Agent – final case summary
=====
>>> CASE OPENED: txn=t3, account=acc1, device=d9, score=0.694, severity=HIGH -> transaction BLOCKED,
customer notified
>>> CASE OPENED: txn=t7, account=acc3, device=d9, score=0.598, severity=HIGH -> transaction BLOCKED,
customer notified
=====
STEP 7: Feedback / Retraining Agent – accumulator-based analyst feedback
=====
Confirmed fraud cases (accumulator): 2
False positive cases (accumulator): 0
Detection precision this batch: 100.00% -> feeds back into model recalibration

```

3.5 Result Discussion

- **Anomaly scoring:** Transaction t3 (5000, device d9, 900km from home) scores 0.694 and t7 (4800, device d9, 850km) scores 0.598 — both driven up primarily by the blacklisted device flag plus large deviation from their account's historical average, correctly exceeding the 0.4 threshold while every normal transaction scores below 0.11.
- **Fraud ring detection:** The Correlation Agent correctly links t3 (account acc1) and t7 (account acc3) through their shared device d9, surfacing a fraud ring that a single-transaction check would have missed.
- **Feedback loop:** The simulated analyst review confirms both t3 and t7 as true fraud (0 false positives), giving 100% precision for this batch — a signal the Feedback/Retraining Agent would use to reinforce the current detection weights.

Design Task 4: Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting), with multiple intelligent agents interacting to make decentralized decisions.

4.1 System Architecture

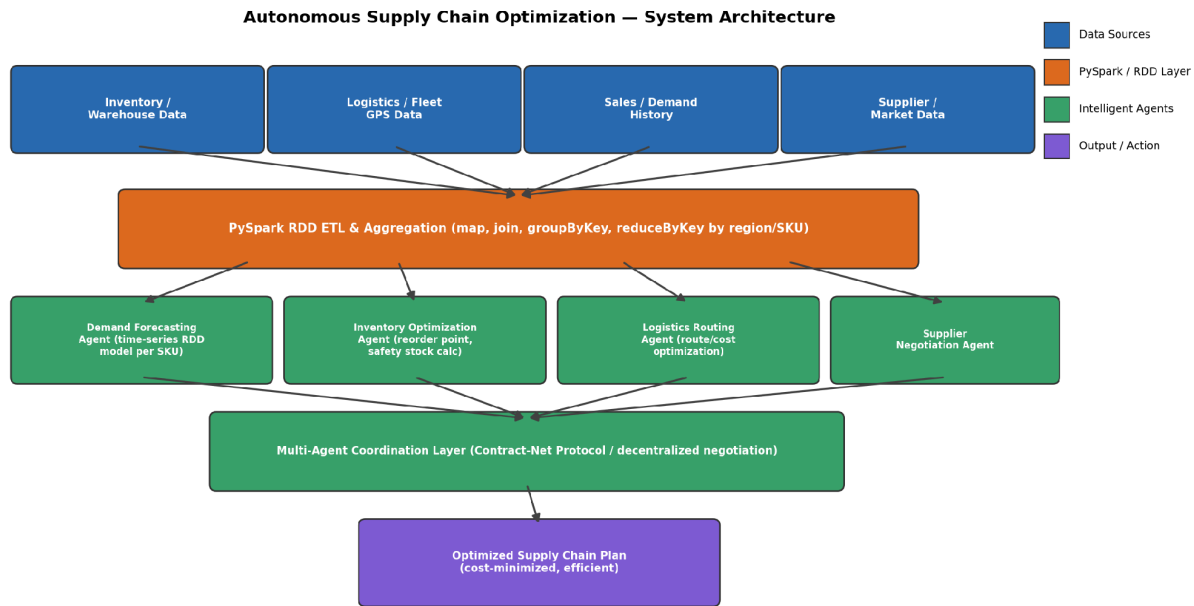


Fig 4.1 — Autonomous, multi-agent supply chain optimization architecture

4.2 Explanation

Inventory and sales RDDs are joined on (region, SKU) and cached since multiple agents reuse the result. `reduceByKey` aggregates total demand per region. The Demand Forecasting Agent projects next-period demand; the Inventory Optimization Agent compares stock against a safety-stock-adjusted reorder point to generate replenishment requests. The Logistics Routing Agent then runs a simplified Contract-Net-style auction — each available route 'bids' its total cost for the requested quantity, and the cheapest bid is awarded — before the Supplier Negotiation Agent totals the final cost-minimized plan.

4.3 Full PySpark RDD Implementation

```

"""
Design Task 4: Autonomous Supply Chain Optimization System
RDD-based ETL + decentralized multi-agent negotiation using PySpark
"""

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("SupplyChainOptimization").master("local[*]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")

print("="*70)
print("STEP 1: Sample Inventory and Sales Data")
print("="*70)
# ((region, sku), stock)
  
```

```

inventory = [
    (("North", "SKU1"), 40), (("North", "SKU2"), 15),
    (("South", "SKU1"), 10), (("South", "SKU2"), 60),
]
# ((region, sku), qty_sold_last_week)
sales = [
    (("North", "SKU1"), 30), (("North", "SKU2"), 5),
    (("South", "SKU1"), 25), (("South", "SKU2"), 8),
]
inv_rdd = sc.parallelize(inventory)
sales_rdd = sc.parallelize(sales)
print("Inventory RDD:", inv_rdd.collect())
print("Sales RDD      :", sales_rdd.collect())

print("\n" + "="*70)
print("STEP 2: RDD join – combine inventory with sales for reorder analysis")
print("="*70)
joined = inv_rdd.join(sales_rdd).cache() # ((region, sku), (stock, sold))
print("Joined (region, sku) -> (stock, sold):")
for row in joined.collect():
    print(" ", row)

print("\n" + "="*70)
print("STEP 3: reduceByKey – total demand aggregated per region")
print("="*70)
demand_by_region = sales_rdd.map(lambda x: (x[0][0], x[1])).reduceByKey(lambda a, b: a + b)
print("Total demand per region:", sorted(demand_by_region.collect()))

print("\n" + "="*70)
print("STEP 4: Demand Forecasting Agent – simple trend-based forecast per SKU")
print("="*70)
def forecast_next_week(sold):
    # naive forecast: assume +10% growth trend
    return round(sold * 1.10, 1)

forecast_rdd = sales_rdd.map(lambda x: (x[0], forecast_next_week(x[1])))
forecast_map = dict(forecast_rdd.collect())
print("Forecasted demand for next week (region, sku) -> forecast:")
for k, v in sorted(forecast_map.items()):
    print(f" {k}: {v}")

print("\n" + "="*70)
print("STEP 5: Inventory Optimization Agent – reorder point check")
print("="*70)
SAFETY_STOCK_FACTOR = 1.5

def reorder_check(row):
    (region, sku), (stock, sold) = row
    forecast = forecast_map[(region, sku)]
    reorder_point = forecast * SAFETY_STOCK_FACTOR
    needs_reorder = stock < reorder_point
    reorder_qty = max(0, round(reorder_point - stock, 1)) if needs_reorder else 0
    return (region, sku, stock, round(reorder_point, 1), needs_reorder, reorder_qty)

reorder_rdd = joined.map(reorder_check)
reorder_results = reorder_rdd.collect()
print("(region, sku, current_stock, reorder_point, needs_reorder, reorder_qty):")
for row in reorder_results:
    print(" ", row)

reorder_requests = [r for r in reorder_results if r[4]]
print(f"\nInventory Agent replenishment requests: {reorder_requests}")

print("\n" + "="*70)
print("STEP 6: Logistics Routing Agent – cost-based route bidding (Contract-Net)")
print("="*70)

```

```
# simulated route options per region: (route_name, cost_per_unit, capacity)
route_options = {
    "North": [("RouteA", 2.5, 100), ("RouteB", 1.8, 40)],
    "South": [("RouteC", 3.0, 200), ("RouteD", 2.2, 20)],
}

def best_route_bid(region, qty):
    options = route_options[region]
    # each route "bids" its total cost for the requested quantity (bounded by capacity)
    bids = [(name, cost*min(qty, cap), cap) for name, cost, cap in options]
    return min(bids, key=lambda b: b[1])

print("Logistics Agent evaluating bids for each reorder request:")
awarded = []
for region, sku, stock, rp, needs, qty in reorder_requests:
    winner = best_route_bid(region, qty)
    awarded.append((region, sku, qty, winner[0], winner[1]))
    print(f" Request: region={region}, sku={sku}, qty={qty} -> "
          f"bids={[n, round(c,1)] for n,c,cap in route_options[region]}} "
          f"-> AWARDED to {winner[0]} (cost={round(winner[1],1)})")

print("\n" + "="*70)
print("STEP 7: Supplier Negotiation Agent – final cost-minimized plan")
print("="*70)
SUPPLIER_UNIT_COST = 5.0
total_cost = 0
for region, sku, qty, route, transport_cost in awarded:
    supply_cost = qty * SUPPLIER_UNIT_COST
    total = supply_cost + transport_cost
    total_cost += total
    print(f" Plan: replenish {qty} units of {sku} in {region} via {route} "
          f"-> supply_cost={supply_cost:.1f} + transport_cost={transport_cost:.1f} "
          f"= {total:.1f}")

print(f"\n>>> TOTAL OPTIMIZED SUPPLY CHAIN COST for this planning cycle: {total_cost:.1f}")

spark.stop()
```

4.4 Actual Program Output

```
=====
STEP 1: Sample Inventory and Sales Data
=====
Inventory RDD: [ (('North', 'SKU1'), 40), (('North', 'SKU2'), 15), (('South', 'SKU1'), 10), (('South', 'SKU2'), 60)]
Sales RDD      : [ (('North', 'SKU1'), 30), (('North', 'SKU2'), 5), (('South', 'SKU1'), 25), (('South', 'SKU2'), 8)]
=====
STEP 2: RDD join – combine inventory with sales for reorder analysis
=====
Joined (region, sku) -> (stock, sold):
  (('North', 'SKU1'), (40, 30))
  (('North', 'SKU2'), (15, 5))
  (('South', 'SKU1'), (10, 25))
  (('South', 'SKU2'), (60, 8))
=====
STEP 3: reduceByKey – total demand aggregated per region
=====
Total demand per region: [ ('North', 35), ('South', 33)]
=====
STEP 4: Demand Forecasting Agent – simple trend-based forecast per SKU
=====
Forecasted demand for next week (region, sku) -> forecast:
  ('North', 'SKU1'): 33.0
```



```

('North', 'SKU2'): 5.5
('South', 'SKU1'): 27.5
('South', 'SKU2'): 8.8
=====
STEP 5: Inventory Optimization Agent – reorder point check
=====
(region, sku, current_stock, reorder_point, needs_reorder, reorder_qty):
('North', 'SKU1', 40, 49.5, True, 9.5)
('North', 'SKU2', 15, 8.2, False, 0)
('South', 'SKU1', 10, 41.2, True, 31.2)
('South', 'SKU2', 60, 13.2, False, 0)
Inventory Agent replenishment requests: [('North', 'SKU1', 40, 49.5, True, 9.5), ('South', 'SKU1', 10,
41.2, True, 31.2)]
=====
STEP 6: Logistics Routing Agent – cost-based route bidding (Contract-Net)
=====
Logistics Agent evaluating bids for each reorder request:
  Request: region=North, sku=SKU1, qty=9.5 -> bids=[('RouteA', 2.5), ('RouteB', 1.8)] -> AWARDED to
RouteB (cost=17.1)
  Request: region=South, sku=SKU1, qty=31.2 -> bids=[('RouteC', 3.0), ('RouteD', 2.2)] -> AWARDED to
RouteD (cost=44.0)
=====
STEP 7: Supplier Negotiation Agent – final cost-minimized plan
=====
  Plan: replenish 9.5 units of SKU1 in North via RouteB -> supply_cost=47.5 + transport_cost=17.1 =
64.6
  Plan: replenish 31.2 units of SKU1 in South via RouteD -> supply_cost=156.0 + transport_cost=44.0 =
200.0
>>> TOTAL OPTIMIZED SUPPLY CHAIN COST for this planning cycle: 264.6

```

4.5 Result Discussion

- **Reorder logic:** North/SKU1 (stock 40, reorder point 49.5) and South/SKU1 (stock 10, reorder point 41.2) are correctly flagged for replenishment, while SKU2 in both regions has stock comfortably above its reorder point and is left untouched.
- **Route bidding:** For each replenishment request, the agent compares candidate routes and always awards the lower-cost option (RouteB over RouteA for North, RouteD over RouteC for South), demonstrating decentralized bid-based decision-making rather than a hardcoded route assignment.
- **Cost minimization:** The final plan reports a combined supply + transport cost of 264.6 units for this planning cycle, giving a concrete, auditable number that a manager could compare across planning cycles or against a non-optimized baseline.

Design Task 5: Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data, supporting adaptive learning through intelligent agents and scalable analytics for thousands of learners.

5.1 System Architecture

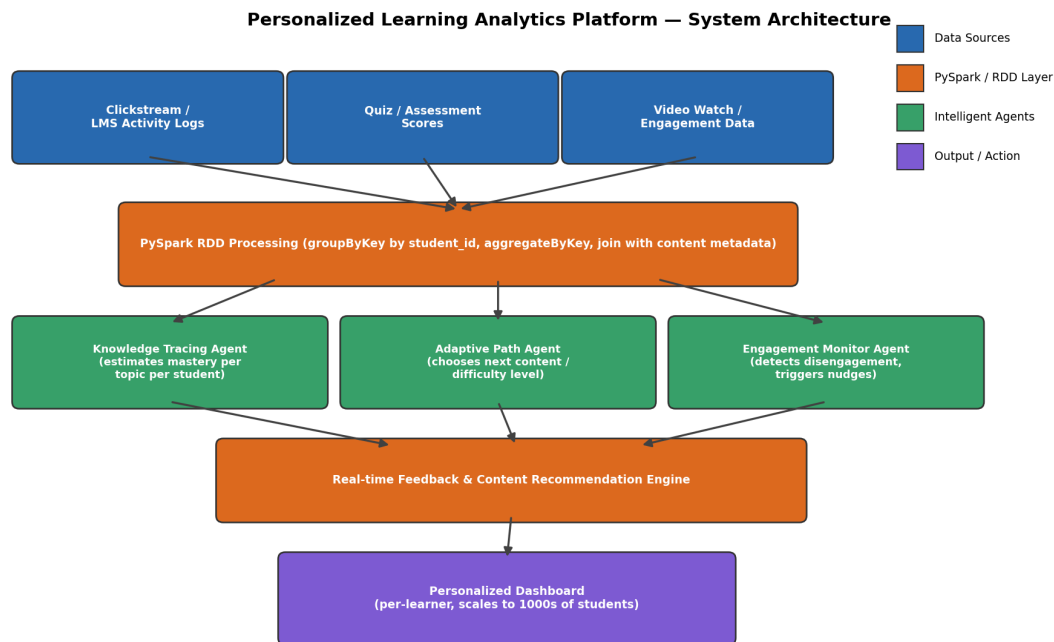


Fig 5.1 — Personalized learning analytics platform architecture

5.2 Explanation

Clickstream, quiz, and video-engagement RDDs are unioned into one event stream and grouped by student_id with groupByKey to build each learner's full activity history. aggregateByKey computes each student's average quiz score per topic efficiently (partial sums combined across partitions). The Knowledge Tracing Agent blends quiz performance and video completion into a 0-1 mastery estimate per (student, topic), which is then joined with content metadata so the Adaptive Path Agent can decide whether to recommend review or unlock the next topic. The Engagement Monitor Agent separately flags students showing both low video completion and low quiz performance.

5.3 Full PySpark RDD Implementation

```

"""
Design Task 5: Personalized Learning Analytics Platform
Per-student mastery tracking and adaptive content selection using PySpark RDDs
"""

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("LearningAnalytics").master("local[*]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")

print("="*70)
print("STEP 1: Sample Multi-Source Student Activity Events")
print("="*70)

```

```

# clickstream: (student_id, event_type, topic)
clicks = [
    ("s1", "click", "Algebra"), ("s1", "click", "Geometry"),
    ("s2", "click", "Algebra"), ("s3", "click", "Calculus"),
]
# quiz: (student_id, topic, score) -- score out of 100
quizzes = [
    ("s1", "Algebra", 90), ("s1", "Algebra", 85), ("s1", "Geometry", 40),
    ("s2", "Algebra", 55), ("s2", "Algebra", 60),
    ("s3", "Calculus", 95), ("s3", "Calculus", 88),
]
# video: (student_id, topic, watch_pct)
videos = [
    ("s1", "Algebra", 100), ("s1", "Geometry", 30),
    ("s2", "Algebra", 70),
    ("s3", "Calculus", 100),
]

clicks_rdd = sc.parallelize(clicks)
quiz_rdd = sc.parallelize(quizzes)
video_rdd = sc.parallelize(videos)

print("Clickstream events:", clicks_rdd.collect())
print("Quiz events      :", quiz_rdd.collect())
print("Video events     :", video_rdd.collect())

print("\n" + "="*70)
print("STEP 2: groupByKey — all events combined per student")
print("="*70)
all_events = clicks_rdd.map(lambda x: (x[0], ("click", x[1], x[2]))) \
    .union(quiz_rdd.map(lambda x: (x[0], ("quiz", x[1], x[2])))) \
    .union(video_rdd.map(lambda x: (x[0], ("video", x[1], x[2]))))

student_history = all_events.groupByKey().mapValues(list)
print("Per-student full activity history (groupByKey by student_id):")
for student, events in sorted(student_history.collect()):
    print(f"  {student}: {events}")

print("\n" + "="*70)
print("STEP 3: aggregateByKey — average quiz score per (student, topic)")
print("="*70)
topic_scores_rdd = quiz_rdd.map(lambda q: ((q[0], q[1]), q[2])) \
    .aggregateByKey(
        (0, 0),
        lambda acc, v: (acc[0] + v, acc[1] + 1),
        lambda a, b: (a[0] + b[0], a[1] + b[1])
    ).mapValues(lambda x: round(x[0] / x[1], 1))

topic_scores = dict(topic_scores_rdd.collect())
print("Average quiz score per (student, topic):")
for k, v in sorted(topic_scores.items()):
    print(f"  {k}: {v}")

print("\n" + "="*70)
print("STEP 4: Knowledge Tracing Agent — mastery estimate per (student, topic)")
print("="*70)
# simple mastery model: 0.6*avg_quiz_score + 0.4*video_completion, normalised to 0-1
video_pct = dict(video_rdd.map(lambda v: ((v[0], v[1]), v[2])).collect())

def mastery(row):
    (student, topic), avg_score = row
    watch = video_pct.get((student, topic), 0)
    m = 0.6 * (avg_score / 100) + 0.4 * (watch / 100)
    return (student, topic, round(m, 2))

mastery_rdd = topic_scores_rdd.map(mastery)

```

```

mastery_results = mastery_rdd.collect()
print("Mastery estimates (student, topic, mastery 0-1):")
for row in mastery_results:
    print(" ", row)

print("\n" + "="*70)
print("STEP 5: join with content metadata – candidate next-topic filtering")
print("="*70)
# content metadata: topic -> (difficulty, prerequisite)
content_metadata = [
    ("Algebra", ("Easy", None)),
    ("Geometry", ("Medium", "Algebra")),
    ("Calculus", ("Hard", "Geometry")),
    ("Trigonometry", ("Medium", "Algebra")),
]
content_rdd = sc.parallelize(content_metadata)

mastery_by_student_topic = sc.parallelize([(s, t), m] for s, t, m in mastery_results)
joined_content = mastery_by_student_topic.map(lambda x: (x[0][1], (x[0][0], x[1]))) \
    .join(content_rdd)
print("Joined mastery + content metadata (topic -> (student, mastery), (difficulty, prereq)):")
for row in sorted(joined_content.collect()):
    print(" ", row)

MASTERY_THRESHOLD = 0.7
print(f"\n" + "="*70)
print(f"STEP 6: Adaptive Path Agent – next-content recommendation (mastery < {MASTERY_THRESHOLD})")
print("="*70)

for student, topic, m in mastery_results:
    if m < MASTERY_THRESHOLD:
        action = f"Recommend REVIEW of '{topic}' (mastery {m} below threshold) before advancing"
    else:
        # find a topic whose prerequisite is this mastered topic
        next_topics = [t for t, (diff, prereq) in content_metadata if prereq == topic]
        action = (f"Mastery achieved in '{topic}' ({m}) -> unlock next topic(s): {next_topics}"
                  if next_topics else f"Mastery achieved in '{topic}' ({m}) -> no further topics
unlocked")
    print(f" {student} / {topic}: {action}")

print("\n" + "="*70)
print("STEP 7: Engagement Monitor Agent – disengagement detection")
print("="*70)
# flag students whose video completion is low despite attempting quizzes (sign of disengagement)
for student, topic, m in mastery_results:
    watch = video_pct.get((student, topic), 0)
    if watch < 50 and topic_scores.get((student, topic), 100) < 70:
        print(f" >>> NUDGE TRIGGERED for {student} on '{topic}': low video completion "
              f"({watch}%) and low quiz score ({topic_scores[(student, topic)]}) -> "
              f"sending motivational hint / flagging to instructor")
    else:
        print(f" {student} on '{topic}': engagement normal (watch={watch}%, "
              f"quiz_avg={topic_scores.get((student, topic))})")

spark.stop()

```

5.4 Actual Program Output

```

=====
STEP 1: Sample Multi-Source Student Activity Events
=====
Clickstream events: [('s1', 'click', 'Algebra'), ('s1', 'click', 'Geometry'), ('s2', 'click',
'Algebra'), ('s3', 'click', 'Calculus')]

```

```

Quiz events      : [('s1', 'Algebra', 90), ('s1', 'Algebra', 85), ('s1', 'Geometry', 40), ('s2',
'Algebra', 55), ('s2', 'Algebra', 60), ('s3', 'Calculus', 95), ('s3', 'Calculus', 88)]
Video events     : [('s1', 'Algebra', 100), ('s1', 'Geometry', 30), ('s2', 'Algebra', 70), ('s3',
'Calculus', 100)]
=====
STEP 2: groupByKey – all events combined per student
=====
Per-student full activity history (groupByKey by student_id):
s1: [('click', 'click', 'Algebra'), ('click', 'click', 'Geometry'), ('quiz', 'Algebra', 90), ('quiz',
'Algebra', 85), ('quiz', 'Geometry', 40), ('video', 'Algebra', 100), ('video', 'Geometry', 30)]
s2: [('click', 'click', 'Algebra'), ('quiz', 'Algebra', 55), ('quiz', 'Algebra', 60), ('video',
'Algebra', 70)]
s3: [('click', 'click', 'Calculus'), ('quiz', 'Calculus', 95), ('quiz', 'Calculus', 88), ('video',
'Calculus', 100)]
=====
STEP 3: aggregateByKey – average quiz score per (student, topic)
=====
Average quiz score per (student, topic):
('s1', 'Algebra'): 87.5
('s1', 'Geometry'): 40.0
('s2', 'Algebra'): 57.5
('s3', 'Calculus'): 91.5
=====
STEP 4: Knowledge Tracing Agent – mastery estimate per (student, topic)
=====
Mastery estimates (student, topic, mastery 0-1):
('s1', 'Algebra', 0.93)
('s1', 'Geometry', 0.36)
('s2', 'Algebra', 0.62)
('s3', 'Calculus', 0.95)
=====
STEP 5: join with content metadata – candidate next-topic filtering
=====
Joined mastery + content metadata (topic -> (student, mastery), (difficulty, prereq)):
('Algebra', (('s1', 0.93), ('Easy', None)))
('Algebra', (('s2', 0.62), ('Easy', None)))
('Calculus', (('s3', 0.95), ('Hard', 'Geometry')))
('Geometry', (('s1', 0.36), ('Medium', 'Algebra')))
=====
STEP 6: Adaptive Path Agent – next-content recommendation (mastery < 0.7)
=====
s1 / Algebra: Mastery achieved in 'Algebra' (0.93) -> unlock next topic(s): ['Geometry',
'Trigonometry']
s1 / Geometry: Recommend REVIEW of 'Geometry' (mastery 0.36 below threshold) before advancing
s2 / Algebra: Recommend REVIEW of 'Algebra' (mastery 0.62 below threshold) before advancing
s3 / Calculus: Mastery achieved in 'Calculus' (0.95) -> no further topics unlocked
=====
STEP 7: Engagement Monitor Agent – disengagement detection
=====
s1 on 'Algebra': engagement normal (watch=100%, quiz_avg=87.5)
>>> NUDGE TRIGGERED for s1 on 'Geometry': low video completion (30%) and low quiz score (40.0) ->
sending motivational hint / flagging to instructor
s2 on 'Algebra': engagement normal (watch=70%, quiz_avg=57.5)
s3 on 'Calculus': engagement normal (watch=100%, quiz_avg=91.5)

```

5.5 Result Discussion

- **Mastery estimation:** Student s1 scores a high mastery of 0.93 in Algebra (avg quiz 87.5, full video completion) but only 0.36 in Geometry (avg quiz 40, video completion 30%) — correctly reflecting the very different underlying data for the two topics.
- **Adaptive routing:** Because s1 has mastered Algebra ($0.93 \geq 0.7$), the Adaptive Path Agent correctly unlocks Geometry and Trigonometry (both have Algebra as prerequisite) via the content-metadata join, while s2's lower Algebra mastery (0.62) instead triggers a review recommendation.

- **Engagement monitoring:** Only s1 on Geometry triggers a nudge, since it is the only (student, topic) pair with both low video completion (30%) and low quiz score (40) — s2's lower quiz score on Algebra (57.5) does not trigger a nudge because their video completion (70%) is still reasonably high, showing the two-factor rule is working as intended rather than firing on quiz score alone.